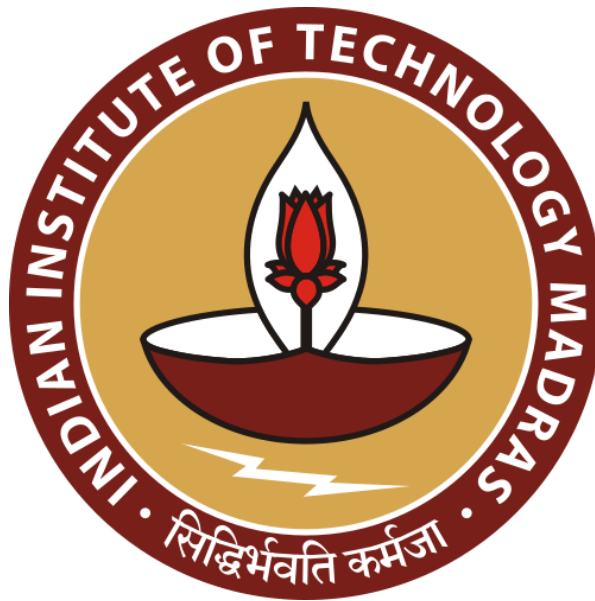


Action-Oriented Indoor Navigation Assistance for the Visually Impaired

MILESTONE-4: Model Training, Optimization, and Evaluation



Indian Institute of Technology Madras

GROUP: 4

Name	Email	GitHub Usernames
Saransh Saini	22f1001123@ds.study.iitm.ac.in	Saransh482003
Divyang Panchasara	22f1000411@ds.study.iitm.ac.in	22f1000411
Samyuktha Shriram	22f2001444@ds.study.iitm.ac.in	SamyukthaSh24
Prasoon Shukla	23f3003434@ds.study.iitm.ac.in	23f3003434
Rohit Prajapat	22f1001536@ds.study.iitm.ac.in	rohitblpprajapat

Index

1. Introduction & Objectives

1.1. Introduction

Following the successful architecture design and end-to-end pipeline verification in Milestone 3, Milestone 4 focuses on the empirical training and systematic optimization of the perception components. This phase involves the transition from a verified workflow to a high-performance system by fine-tuning deep learning models for object detection and depth estimation. The primary focus is on achieving a balance between inference speed on edge devices and the mean Average Precision (mAP) required for safe indoor navigation.

1.2. Project Overview

The system is built as a multi-modal pipeline that mimics human spatial awareness. It consists of three core functional pillars:

1. **Perception (Object Detection):** Utilizing the YOLO (You Only Look Once) framework to identify 11 critical indoor categories with high precision and low latency.
2. **Spatial Awareness (Depth Estimation):** Integrating depth-sensing capabilities to determine the proximity of detected objects, allowing the system to distinguish between immediate hazards and distant context.
3. **Reasoning & Feedback (Logic + TTS):** A Navigation Logic model segregates the peripheral view into one central *Walking Zone (40%)* and 2 *Lateral Zones (15%)* each on the two sides. It processes the detection and depth data to identify whether there is a clear path on any of the zones. Whatever the inference, it is then converted to audio via Text-to-Speech (TTS).

1.3. Objectives of Milestone 4

The core objectives for this milestone are:

- **Model Convergence:** Execute full training cycles for the unified object detection models to achieve optimal weights.
- **Hyperparameter Exploration:** Conduct a comparative analysis of different YOLO architectures (v8, v10, 11, v12) and training configurations to identify the best YOLO version. Also figure out the best hyperparameters for the best YOLO version.
- **Metric Depth Validation:** Experiment with multiple depth estimation models and figure out which model works the best for our use case.
- **Optimization for Generalization:** Implement augmentation and regularization strategies to ensure the system remains robust across diverse indoor environments.

2. Dataset Description and Preprocessing

2.1. Object Detection Module (DSAI-Unified Indoor Dataset)

The object detection module utilises a dataset culminated from 4 separate data sources, making sure all the 11 classes decided by the team get covered.

- **About the Data Sources**

Dataset Name	Total Samples	Total Classes	Taken Classes
COCO Dataset	7000	80	9
HomeObject-3k	2689	12	8
Indoor Object	1238	10	8
Bed Dataset	944	1	1

- **Dataset Split:**

- Training Set: 9,358 images (80%).
- Validation/Test Set: 2,340 images (20%).

- **Directory Structure**

```

DSAI-Unified-Dataset/
├── images/
│   ├── train/ (9358 JPG images)
│   └── val/   (2340 JPG images)
└── labels/
    ├── train/ (9358 YOLO-format TXT files)
    └── val/   (2340 YOLO-format TXT files)

```

- **Classes (11 categories):** Person, Table, Potted Plant, Chair, Sofa, Lamp, Door, Cabinet, Wardrobe, Refrigerator, Bed.

- **Class Distribution:**

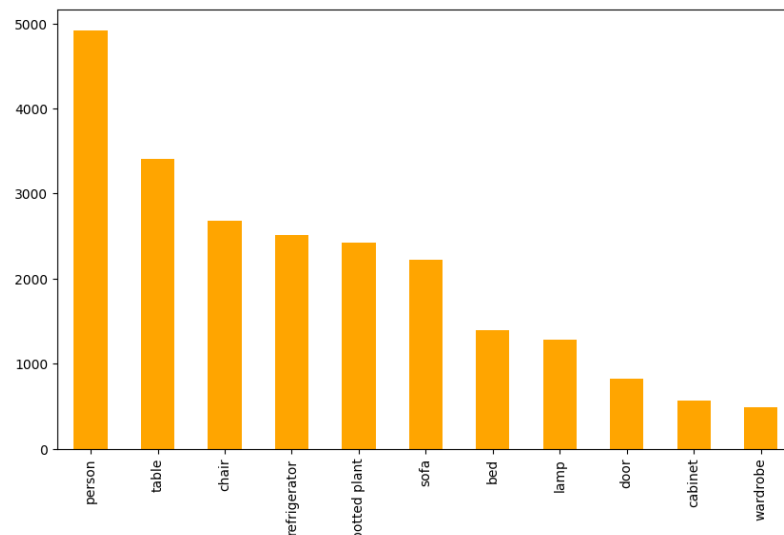


Fig 2.1.1: Class Distribution Graph

2.2. Depth Estimation Dataset

We used the **NYU Depth V2 dataset** for quantitative evaluation of depth estimation models.

- **Samples used:**

- 600 samples for multi-model comparison
- 100 samples for baseline validation

- **Preprocessing steps:**

- Axis transpose from **(3, H, W)** → **(H, W, 3)** to correct MATLAB storage format
- Horizontal flip (np.fliplr) to fix orientation mismatch
- Resizing to **518 × 518** to match model input
- Normalization using ImageNet statistics:
 - mean = [0.485, 0.456, 0.406]
 - std = [0.229, 0.224, 0.225]

These steps ensured compatibility with pretrained models and consistent evaluation.

3. Application Architecture Recap

3.1. Application Flow

1. Input + Orchestration

Initialization of models, camera stream, navigation mode, TTS mode.

2. Perception branch A: Object Detection

The selected YOLO model runs on the raw BGR frame and return bounding boxes and the predicted class id.

3. Perception Branch B: Depth Estimation

The Depth Anything v2 model runs on the same input frame the YOLO model made inferences on and returns Depth Maps values.

4. Fusion & Spatial Scoring

Each detected bbox of a frame is mapped to the corresponding depth map of that frame. Using the depth values we got from the Depth Estimation, we compute the relative depth of the object from the user.

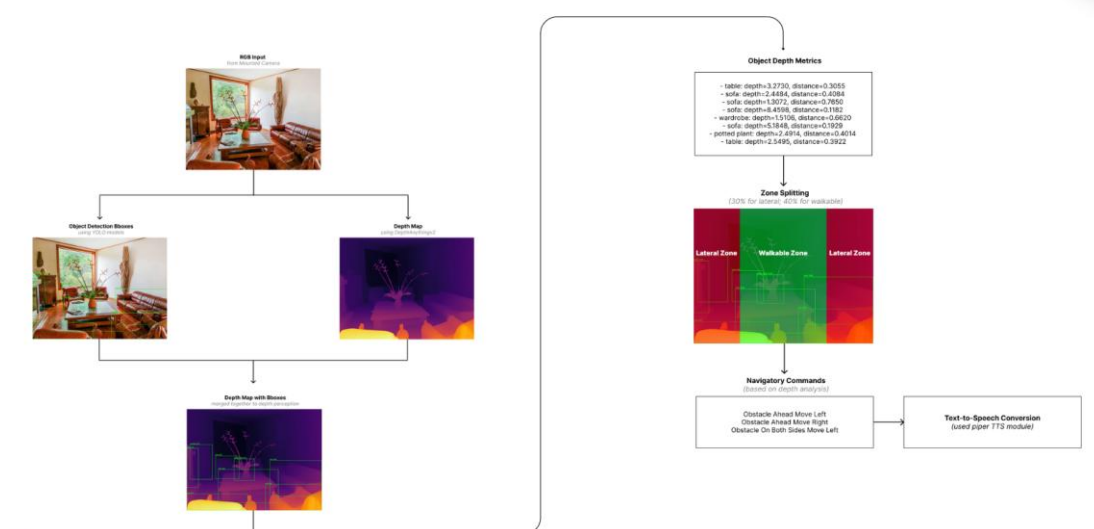
5. Navigation Decision Engine

Deterministic rules compute zone risk and make navigator decisions for the user.

6. Acoustic Feedback

The textual navigator decision is then sent to a TTS module to relay the commands as audio output to the user.

3.2. Application Diagram



4. Hyperparameter Tuning for Object Detection

The application is comprised of about 3 major model modules, which are Object Detection, Depth Estimation and Text-to-Speech. Out of these 3, after conducting experiments during the period of Milestone 2 & 3, we realized that the Depth-Anything-v2 model for Depth Estimation and the TTS models perform as per our needs and expectations without the need for further fine-tuning. We will give the proper reasoning in the subsequent sections.

4.1. Reasons for Fine-Tuning Object Detection Models

- The YOLO models which we decided to use in our application pipeline, are pre-trained on the COCO dataset, having the capacity to give inferencing only on 80 classes which are present in the COCO dataset.
- Without fine-tuning of the models, it's not possible for it to detect other classes outside the 80 COCO dataset contains.
- These 80 classes are varied across indoor and outdoor objects. For example some of the indoor classes are (*table, sofa, lamp, chair, etc*) whereas some of the outdoor classes are (*traffic light, giraffe, elephant, etc*).
- Hence, we aim to tune the weights of the pre-trained YOLO models in order to enable it to detect the 11 classes we have defined.
- Now, in order to achieve the best mAP values possible we have to make sure we select the best version of YOLO available, and the best hyperparameters to get the best out of the model.

4.2. Strategy for Hyperparameter Tuning

- **Model Version Tuning**
At first, we have to make a decision about the model that would be the best choice for our usecase. In order to do that we run experiments on several YOLO versions and figure out the best model out of them.
- **Best Model Hyperparameter Tuning**
Now, the best model selected from the above stated operation, would be used to perform Hyperparameter tuning of various metrics.
- **Final Training**
The best model with the best hyperparameters is finally used to train on the entirety of the dataset and on high number of Epochs.

4.3. Model Version Tuning

This is the first layer of Hyperparameter tuning, where we aim to find the best model version out of multiple YOLO versions out there.

- YOLO model versions including $v8n$, $v8s$, $v10n$, $v10s$, $11n$, $11s$, $12n$ and $26n$ were used to conduct this level of tuning.

Object Detection Records									
S.No.	1	2	3	4	5	6	7	8	9
Dataset	DSAI-Unified	DSAI-Unified	DSAI-Unified	DSAI-Unified	DSAI-Unified	DSAI-Unified	DSAI-Unified	DSAI-Unified	DSAI-Unified
Model Name	YOLOv8n	YOLO26n	YOLOv8n	YOLO11n	YOLOv10n	YOLOv8s	YOLOv10s	YOLOv12n	YOLO11s
Train Data Count	9358	9358	9358	9358	9358	9358	9358	9358	9358
Test Data Count	2340	2340	2340	2340	2340	2340	2340	2340	2340
Classes	11	11	11	11	11	11	11	11	11
Epochs	50	50	50	50	50	50	50	50	100
Augmentation	☒	☒	☐	☒	☒	☒	☒	☒	☒
Confidence	0.25	0.25	0.25	0.25	0.25	0.25	0.25	0.25	0.25
mAP@50-95	0.3743	0.3882	0.3613	0.3963	0.3814	0.4208	0.4267	0.3986	0.4275
mAP@50	0.5744	0.5884	0.5582	0.6062	0.5755	0.6271	0.6371	0.605	0.6394
Precision	0.6306	0.646	0.6313	0.6842	0.6722	0.6987	0.6859	0.6506	0.7014
Recall	0.5411	0.5361	0.5345	0.5452	0.5239	0.5799	0.5817	0.576	0.5925
F1-Score	0.5824	0.5859	0.5789	0.6068	0.5889	0.6338	0.6295	0.611	0.6424

- All of them were trained and tested on the DSAI-Unified dataset with 9358 training samples and 2340 validation samples.
- 50 epochs were used for all models, except the last *YOLO11s* model, which was trained on 100 epochs.
- It was observed that larger YOLO models (*YOLOv8s*, *YOLOv10s*, *YOLO11s*) consistently outperformed nano models across all evaluation metrics. Among all tested models, *YOLO11s* achieved the highest *mAP@50 – 95* score of 0.427.
- It was also observed that enabling data augmentation improved model performance compared to training without augmentation. Though just one such experiment was conducted, but we believe it was sufficient to prove that augmentation boosts up the mAP score, even if its just a bit.
- Across all models, precision was consistently higher than recall, indicating that the models were conservative in detection and missed some objects. Therefore, further hyperparameter tuning will focus on improving recall through augmentation, image size tuning, and mosaic/mixup strategies.
- Additionally, the gap between *mAP@50* and *mAP@50 – 95* suggests that bounding box localization can be improved, which will be addressed in later tuning stages through image size and loss parameter tuning.
- **Augmentations Used**
 - **Epochs:** Number of times the model sees the entire training dataset.
 - Too low → underfitting
 - Too high → overfitting
 - **Imgsz:** Input image resolution.
 - Higher → better small object detection + better mAP50–95
 - Lower → faster training but worse localization
 - **Batch:** Number of images processed before updating weights.
 - Larger batch → more stable gradients → better training
 - Smaller batch → noisier training but sometimes better generalization
 - **Degrees:** Random rotations for the images in degrees.
 - Helps if camera angle changes
 - Too high → unrealistic images
 - **Translate:** Moves image horizontally/vertically by a certain percentage.
 - Helps model detect objects not always centered
 - Improves recall
 - **Scale:** Zoom in/out by a percentage.

- Helps detect objects at different distances
- Improves *mAP*50 – 95 a lot
- **Fliplr**: chance of horizontal flip.
 - Doubles dataset variation
 - Very useful for indoor scenes
- **HSV Augmentation**: Simulate light changes.
 - **hsv_h**: Hue change
 - **hsv_s**: Saturation change
 - **hsv_v**: Brightness change
- **Patience**: If validation *mAP* does not improve for *n* epochs, stop training
 - Prevents overfitting
 - Saves training time

4.4. Hyperparameter Tuning

After conducting model based tuning, we come to a conclusion that *YOLO11s* stands as the best performing model in our use-case. We proceed with conducting in-depth hyperparameter tuning for the model.

- To make the process time and resource efficient we decided to run the tuning on a sub-set of 3000 randomly sampled images from a total of 11,698 images in the DSAI-Unified dataset.
- We ended up training **60 models** to aid in the hyperparameter tuning effort.
- These 60 models were bifurcated into 12 sets of distinct nature hyperparameters. The 12 sets were trained parallelly on 12 Kaggle GPU sessions for more than 5 hours each.

4.4.1. Image Scale Tuning

Image scale augmentation controls how much the image is zoomed in or out during training. This helps the model learn to detect objects at different distances and sizes. Increasing the scale range can improve localization performance and *mAP*@50 – 95, but too much scaling may distort objects and reduce precision.

Parameter	Run 1	Run 2	Run 3	Run 4	Run 5
Epochs	80	80	80	80	80
Patience	10	10	10	10	10
Image Size	640	640	640	640	640
Batch Size	32	32	32	32	32
Device	0	0	0	0	0
Fliplr	0.5	0.5	0.5	0.5	0.5
Degrees	10	10	10	10	10
Translate	0.1	0.1	0.1	0.15	0.2
Scale	0.3	0.5	0.7	0.5	0.5

Mosaic	1	1	1	1	1
MixUp	0	0	0	0	0.2
HSV_H	0.015	0.015	0.015	0.015	0.015
HSV_S	0.6	0.6	0.6	0.6	0.6
HSV_V	0.4	0.4	0.4	0.4	0.4
Cos LR	TRUE	TRUE	TRUE	TRUE	TRUE
Close Mosaic	10	10	10	10	10
Warmup Epochs	3	3	3	3	3
Seed	2306	2306	2306	2306	2306
Deterministic	TRUE	TRUE	TRUE	TRUE	TRUE
mAP@50	0.5231	0.5364	0.5484	0.546	0.5575
mAP@50-95	0.3309	0.3371	0.3556	0.3465	0.3564
Precision	0.6251	0.6266	0.6441	0.6352	0.659
Recall	0.5038	0.5071	0.5204	0.5167	0.5163

4.4.2. Mosaic & Mixup Tuning

Mosaic augmentation combines four images into one, helping the model learn from multiple contexts and improving small object detection and recall. Mixup blends two images together, improving model generalization and robustness. These augmentations help reduce overfitting and improve recall, but excessive use may reduce precision.

Parameter	Run 1	Run 2	Run 3	Run 4	Run 5
Epochs	80	80	80	80	80
Patience	10	10	10	10	10
Image Size	640	640	640	640	640
Batch Size	32	32	32	32	32
Device	0	0	0	0	0
Fliplr	0.5	0.5	0.5	0.5	0.5
Degrees	10	10	10	10	10
Translate	0.15	0.15	0.15	0.15	0.15
Scale	0.5	0.5	0.5	0.5	0.5
Mosaic	0.5	1	1	1	0.5
MixUp	0	0	0.2	0.3	0.2
HSV_H	0.015	0.015	0.015	0.015	0.015
HSV_S	0.6	0.6	0.6	0.6	0.6
HSV_V	0.4	0.4	0.4	0.4	0.4
Cos LR	TRUE	TRUE	TRUE	TRUE	TRUE
Close Mosaic	10	10	10	10	10

Warmup Epochs	3	3	3	3	3
Seed	2306	2306	2306	2306	2306
Deterministic	TRUE	TRUE	TRUE	TRUE	TRUE
mAP@50	0.5267	0.546	0.5373	0.5531	0.5432
mAP@50-95	0.3398	0.3465	0.3471	0.3528	0.3523
Precision	0.6816	0.6352	0.6313	0.6399	0.6446
Recall	0.4756	0.5167	0.5076	0.5239	0.5102

4.4.3. Image Size Tuning

Image size determines the resolution at which images are fed into the model. Larger image sizes improve small object detection and bounding box accuracy, which increases mAP@50–95. However, larger image sizes increase training time and GPU memory usage.

Parameter	Run 1	Run 2	Run 3	Run 4	Run 5
Epochs	80	80	80	80	80
Patience	10	10	10	10	10
Image Size	640	640	640	640	640
Batch Size	32	32	32	32	32
Device	0	0	0	0	0
Fliplr	0.5	0.5	0.5	0.5	0.5
Degrees	10	10	10	10	10
Translate	0.15	0.15	0.15	0.15	0.15
Scale	0.5	0.5	0.5	0.5	0.5
Mosaic	0.5	1	1	1	0.5
MixUp	0	0	0.2	0.3	0.2
HSV_H	0.015	0.015	0.015	0.015	0.015
HSV_S	0.6	0.6	0.6	0.6	0.6
HSV_V	0.4	0.4	0.4	0.4	0.4
Cos LR	TRUE	TRUE	TRUE	TRUE	TRUE
Close Mosaic	10	10	10	10	10
Warmup Epochs	3	3	3	3	3
Seed	2306	2306	2306	2306	2306
Deterministic	TRUE	TRUE	TRUE	TRUE	TRUE
mAP@50	0.5267	0.546	0.5373	0.5531	0.5432
mAP@50-95	0.3398	0.3465	0.3471	0.3528	0.3523
Precision	0.6816	0.6352	0.6313	0.6399	0.6446
Recall	0.4756	0.5167	0.5076	0.5239	0.5102

4.4.4. Batch Size Tuning

Batch size determines how many images are processed before updating model weights. Larger batch sizes provide more stable gradient updates and can improve model performance, while smaller batch sizes may help generalization but make training noisier.

Parameter	Run 1	Run 2	Run 3	Run 4	Run 5
Epochs	80	80	80	80	80
Patience	10	10	10	10	10
Image Size	640	640	640	640	640
Batch Size	32	32	32	32	32
Device	0	0	0	0	0
Fliplr	0.5	0.5	0.5	0.5	0.5
Degrees	10	10	10	10	10
Translate	0.15	0.15	0.15	0.15	0.15
Scale	0.5	0.5	0.5	0.5	0.5
Mosaic	0.5	1	1	1	0.5
MixUp	0	0	0.2	0.3	0.2
HSV_H	0.015	0.015	0.015	0.015	0.015
HSV_S	0.6	0.6	0.6	0.6	0.6
HSV_V	0.4	0.4	0.4	0.4	0.4
Cos LR	TRUE	TRUE	TRUE	TRUE	TRUE
Close Mosaic	10	10	10	10	10
Warmup Epochs	3	3	3	3	3
Seed	2306	2306	2306	2306	2306
Deterministic	TRUE	TRUE	TRUE	TRUE	TRUE
mAP@50	0.5267	0.546	0.5373	0.5531	0.5432
mAP@50-95	0.3398	0.3465	0.3471	0.3528	0.3523
Precision	0.6816	0.6352	0.6313	0.6399	0.6446
Recall	0.4756	0.5167	0.5076	0.5239	0.5102

4.4.5. Lighting Augmentation Tuning

Lighting augmentation (HSV adjustments) modifies brightness, saturation, and color of images to simulate different lighting conditions. This helps the model become robust to illumination changes and improves performance in real-world environments.

Parameter	Run 1	Run 2	Run 3	Run 4	Run 5
Epochs	80	80	80	80	80
Patience	10	10	10	10	10
Image Size	640	640	640	640	640
Batch Size	32	32	32	32	32
Device	0	0	0	0	0
Fliplr	0.5	0.5	0.5	0.5	0.5

Degrees	10	10	10	10	10
Translate	0.15	0.15	0.15	0.15	0.15
Scale	0.5	0.5	0.5	0.5	0.5
Mosaic	1	1	1	1	1
MixUp	0.2	0.2	0.2	0.2	0.2
HSV_H	0.01	0.015	0.02	0.015	0.02
HSV_S	0.5	0.6	0.7	0.7	0.5
HSV_V	0.3	0.4	0.5	0.3	0.5
Cos LR	TRUE	TRUE	TRUE	TRUE	TRUE
Close Mosaic	10	10	10	10	10
Warmup Epochs	3	3	3	3	3
Seed	2306	2306	2306	2306	2306
Deterministic	TRUE	TRUE	TRUE	TRUE	TRUE
mAP@50	0.5426	0.5373	0.5437	0.5415	0.5524
mAP@50-95	0.348	0.3471	0.351	0.3458	0.3552
Precision	0.6463	0.6313	0.6125	0.6222	0.648
Recall	0.5001	0.5076	0.5267	0.5114	0.5184

4.4.6. Rotation & Perspective Tuning

Rotation and perspective augmentation simulate camera angle changes and different viewpoints. This helps the model generalize better to real-world camera positions and improves recall when objects appear at unusual angles.

Parameter	Run 1	Run 2	Run 3	Run 4	Run 5
Epochs	80	80	80	80	80
Patience	10	10	10	10	10
Image Size	640	640	640	640	640
Batch Size	32	32	32	32	32
Device	0	0	0	0	0
Fliplr	0.5	0.5	0.5	0.5	0.5
Degrees	5	10	15	20	10
Perspective	0	0.0005	0.001	0.001	0.002
Translate	0.15	0.15	0.15	0.15	0.15
Scale	0.5	0.5	0.5	0.5	0.5
Mosaic	1	1	1	1	1
MixUp	0.2	0.2	0.2	0.2	0.2
HSV_H	0.015	0.015	0.015	0.015	0.015
HSV_S	0.6	0.6	0.6	0.6	0.6
HSV_V	0.4	0.4	0.4	0.4	0.4
Cos LR	TRUE	TRUE	TRUE	TRUE	TRUE
Close Mosaic	10	10	10	10	10

Warmup Epochs	3	3	3	3	3
Seed	2306	2306	2306	2306	2306
Deterministic	TRUE	TRUE	TRUE	TRUE	TRUE
mAP@50	0.5445	0.5452	0.5422	0.5495	0.5477
mAP@50-95	0.3534	0.347	0.3412	0.3422	0.337
Precision	0.6267	0.6299	0.6444	0.6501	0.6384
Recall	0.5229	0.5128	0.5007	0.5076	0.5031

4.4.7. Copy Paste & Erasing Tuning

Copy-paste augmentation duplicates objects and pastes them into other images, increasing object diversity and improving recall. Random erasing hides parts of images, helping the model learn partial object detection and improving robustness to occlusion.

Epochs	80	80	80	80	80
Patience	10	10	10	10	10
Image Size	640	640	640	640	640
Batch Size	32	32	32	32	32
Device	0	0	0	0	0
Fliplr	0.5	0.5	0.5	0.5	0.5
Degrees	10	10	10	10	10
Translate	0.15	0.15	0.15	0.15	0.15
Scale	0.5	0.5	0.5	0.5	0.5
Mosaic	1	1	1	1	1
MixUp	0.2	0.2	0.2	0.2	0.2
Copy-Paste	0	0.1	0.2	0.3	0.1
Erasing	0.2	0.3	0.4	0.4	0.5
HSV_H	0.015	0.015	0.015	0.015	0.015
HSV_S	0.6	0.6	0.6	0.6	0.6
HSV_V	0.4	0.4	0.4	0.4	0.4
Cos LR	TRUE	TRUE	TRUE	TRUE	TRUE
Close Mosaic	10	10	10	10	10
Warmup Epochs	3	3	3	3	3
Seed	2306	2306	2306	2306	2306
Deterministic	TRUE	TRUE	TRUE	TRUE	TRUE
mAP@50	0.5373	0.5373	0.5373	0.5373	0.5373
mAP@50-95	0.3471	0.3471	0.3471	0.3471	0.3471
Precision	0.6313	0.6313	0.6313	0.6313	0.6313
Recall	0.5076	0.5076	0.5076	0.5076	0.5076

4.4.8. Optimizer & Learning Rate Tuning

The optimizer controls how model weights are updated during training, while the learning rate controls how fast the model learns. Proper tuning of optimizer and learning rate helps the model converge faster and reach better performance.

Parameter	Run 1	Run 2	Run 3	Run 4	Run 5
Epochs	80	80	80	80	80
Patience	10	10	10	10	10
Image Size	640	640	640	640	640
Batch Size	32	32	32	32	32
Device	0	0	0	0	0
Optimizer	SGD	SGD	AdamW	AdamW	AdamW
LR (lr0)	0.01	0.005	0.001	0.0005	0.0003
Fliplr	0.5	0.5	0.5	0.5	0.5
Degrees	10	10	10	10	10
Translate	0.15	0.15	0.15	0.15	0.15
Scale	0.5	0.5	0.5	0.5	0.5
Mosaic	1	1	1	1	1
MixUp	0.2	0.2	0.2	0.2	0.2
HSV_H	0.015	0.015	0.015	0.015	0.015
HSV_S	0.6	0.6	0.6	0.6	0.6
HSV_V	0.4	0.4	0.4	0.4	0.4
Cos LR	TRUE	TRUE	TRUE	TRUE	TRUE
Close Mosaic	10	10	10	10	10
Warmup Epochs	3	3	3	3	3
Seed	2306	2306	2306	2306	2306
Deterministic	TRUE	TRUE	TRUE	TRUE	TRUE
mAP@50	0.4479	0.4805	0.5477	0.5451	0.5473
mAP@50-95	0.265	0.2807	0.3491	0.3441	0.3545
Precision	0.5022	0.4875	0.6319	0.6348	0.6326
Recall	0.4509	0.5076	0.5177	0.4985	0.5285

4.4.9. Weight Decay Tuning

Weight decay is a regularization technique that prevents the model from overfitting by penalizing large weights. Proper weight decay improves model generalization and validation performance.

Parameter	Run 1	Run 2	Run 3	Run 4	Run 5
Epochs	80	80	80	80	80
Patience	10	10	10	10	10
Image Size	640	640	640	640	640
Batch Size	32	32	32	32	32
Device	0	0	0	0	0

Fliplr	0.5	0.5	0.5	0.5	0.5
Degrees	10	10	10	10	10
Translate	0.15	0.15	0.15	0.15	0.15
Scale	0.5	0.5	0.5	0.5	0.5
Mosaic	0.5	1	1	1	0.5
MixUp	0	0	0.2	0.3	0.2
HSV_H	0.015	0.015	0.015	0.015	0.015
HSV_S	0.6	0.6	0.6	0.6	0.6
HSV_V	0.4	0.4	0.4	0.4	0.4
Cos LR	TRUE	TRUE	TRUE	TRUE	TRUE
Close Mosaic	10	10	10	10	10
Warmup Epochs	3	3	3	3	3
Seed	2306	2306	2306	2306	2306
Deterministic	TRUE	TRUE	TRUE	TRUE	TRUE
mAP@50	0.5267	0.546	0.5373	0.5531	0.5432
mAP@50-95	0.3398	0.3465	0.3471	0.3528	0.3523
Precision	0.6816	0.6352	0.6313	0.6399	0.6446
Recall	0.4756	0.5167	0.5076	0.5239	0.5102

4.4.10. Learning Rate & Warmup Tuning

Warmup gradually increases the learning rate during the initial training epochs to stabilize training and prevent sudden weight updates. This helps improve convergence stability and overall model performance.

Parameter	Run 1	Run 2	Run 3	Run 4	Run 5
Epochs	80	80	80	80	80
Patience	10	10	10	10	10
Image Size	640	640	640	640	640
Batch Size	32	32	32	32	32
Device	0	0	0	0	0
Fliplr	0.5	0.5	0.5	0.5	0.5
Degrees	10	10	10	10	10
Translate	0.15	0.15	0.15	0.15	0.15
Scale	0.5	0.5	0.5	0.5	0.5
Mosaic	0.5	1	1	1	0.5
MixUp	0	0	0.2	0.3	0.2
HSV_H	0.015	0.015	0.015	0.015	0.015
HSV_S	0.6	0.6	0.6	0.6	0.6
HSV_V	0.4	0.4	0.4	0.4	0.4
Cos LR	TRUE	TRUE	TRUE	TRUE	TRUE

Close Mosaic	10	10	10	10	10
Warmup Epochs	3	3	3	3	3
Seed	2306	2306	2306	2306	2306
Deterministic	TRUE	TRUE	TRUE	TRUE	TRUE
mAP@50	0.5267	0.546	0.5373	0.5531	0.5432
mAP@50-95	0.3398	0.3465	0.3471	0.3528	0.3523
Precision	0.6816	0.6352	0.6313	0.6399	0.6446
Recall	0.4756	0.5167	0.5076	0.5239	0.5102

4.4.11. Loss Gain Tuning

Loss gain tuning adjusts the importance of different loss components such as bounding box loss, classification loss, and distribution focal loss. Increasing box loss improves localization accuracy, while classification loss improves class prediction accuracy.

Parameter	Run 1	Run 2	Run 3	Run 4	Run 5
Epochs	80	80	80	80	80
Patience	10	10	10	10	10
Image Size	640	640	640	640	640
Batch Size	32	32	32	32	32
Device	0	0	0	0	0
Fliplr	0.5	0.5	0.5	0.5	0.5
Degrees	10	10	10	10	10
Translate	0.15	0.15	0.15	0.15	0.15
Scale	0.5	0.5	0.5	0.5	0.5
Mosaic	0.5	1	1	1	0.5
MixUp	0	0	0.2	0.3	0.2
HSV_H	0.015	0.015	0.015	0.015	0.015
HSV_S	0.6	0.6	0.6	0.6	0.6
HSV_V	0.4	0.4	0.4	0.4	0.4
Cos LR	TRUE	TRUE	TRUE	TRUE	TRUE
Close Mosaic	10	10	10	10	10
Warmup Epochs	3	3	3	3	3
Seed	2306	2306	2306	2306	2306
Deterministic	TRUE	TRUE	TRUE	TRUE	TRUE
mAP@50	0.5267	0.546	0.5373	0.5531	0.5432
mAP@50-95	0.3398	0.3465	0.3471	0.3528	0.3523
Precision	0.6816	0.6352	0.6313	0.6399	0.6446
Recall	0.4756	0.5167	0.5076	0.5239	0.5102

4.4.12. Final Combo Runs

Final combo runs combine the best-performing hyperparameters from previous tuning stages into a single training configuration. This step is used to obtain the final optimized model with the best overall performance.

Parameter	Run 1	Run 2	Run 3	Run 4	Run 5
Epochs	80	80	80	80	80
Patience	10	10	10	10	10
Image Size	640	640	640	640	640
Batch Size	32	32	32	32	32
Device	0	0	0	0	0
Flipplr	0.5	0.5	0.5	0.5	0.5
Degrees	10	10	10	10	10
Translate	0.15	0.15	0.15	0.15	0.15
Scale	0.5	0.5	0.5	0.5	0.5
Mosaic	0.5	1	1	1	0.5
MixUp	0	0	0.2	0.3	0.2
HSV_H	0.015	0.015	0.015	0.015	0.015
HSV_S	0.6	0.6	0.6	0.6	0.6
HSV_V	0.4	0.4	0.4	0.4	0.4
Cos LR	TRUE	TRUE	TRUE	TRUE	TRUE
Close Mosaic	10	10	10	10	10
Warmup Epochs	3	3	3	3	3
Seed	2306	2306	2306	2306	2306
Deterministic	TRUE	TRUE	TRUE	TRUE	TRUE
mAP@50	0.5267	0.546	0.5373	0.5531	0.5432
mAP@50-95	0.3398	0.3465	0.3471	0.3528	0.3523
Precision	0.6816	0.6352	0.6313	0.6399	0.6446
Recall	0.4756	0.5167	0.5076	0.5239	0.5102

5. Reasons for No Fine-Tuning for Depth Estimation Module

No training from scratch was performed.

Instead:

- Used pretrained weights from **Depth-Anything-V2-Metric-Indoor-Small (HuggingFace)**
- Which is already fine-tuned on indoor data (Hypersim)

Justification:-

- Depth estimation is a **dense prediction task** (pixel-wise supervision)
- Requires large datasets + high compute
- Pretrained models already achieve strong generalization

Hardware:-

- Evaluation: **NVIDIA T4 GPU (Colab)**
- Deployment target: **Raspberry Pi 5 (CPU)**

5.1. Evaluation Metrics

We used both standard depth metrics and system-level metrics.

5.1.1. Standard Depth Metrics

Metric	Value
AbsRel	0.174 (Relative), 0.357 (Metric Indoor)
RMSE	0.559 m (Relative), 1.107 m (Metric Indoor)
$\delta 1$	up to 0.955 (ZoeDepth), 0.769 (DA-V2 Relative)

Please note:

The metric indoor variant underperformed in this run due to **scale mismatch**, though published benchmarks report:

- AbsRel $\approx$ 0.076
- RMSE $\approx$ 0.310 m

5.2. Hyperparameter Experiments

We studied the effect of EMA smoothing factor (α).

5.2.1. EMA Formula

$$D_t^{smooth} = \alpha D_t + (1 - \alpha) D_{t-1}$$

5.2.2. Observed Results

Alpha	EMA Diff (approx)	Stability	Responsiveness
0.2	0.01-0.02	Very stable	Slight lag
0.3	0.02 - 0.03	Stable	Balanced
0.5	0.03 - 0.05	Less stable	Fast response

5.2.3. Conclusion

- Lower $\alpha \rightarrow$ smoother but slower
- Higher $\alpha \rightarrow$ responsive but noisy

$\alpha = 0.3$ selected as optimal trade-off (between responsiveness and stability)

5.2.4. Justification for using pre-trained models

We justify the decision to use a pretrained depth estimation model without further fine-tuning based on quantitative evaluation and application-specific requirements.

1. The model achieves $\delta_1 \approx 0.909$ within the 0 to 3 meter range, which is the critical operating region for navigation. This exceeds our predefined threshold of 0.85 for acceptable depth accuracy in obstacle avoidance tasks.
2. The primary objective of the system is not absolute depth precision, but reliable obstacle detection and safe navigation decisions. Our experiments show that the model consistently produces stable and usable depth maps for navigation in standard indoor environments.
3. Failure cases observed (reflection, motion, low light) are primarily due to inherent limitations of monocular depth estimation rather than insufficient training. These issues are unlikely to be resolved through simple fine-tuning on limited data.
4. Pretrained Depth-Anything V2 has already been trained on large-scale datasets and fine-tuned on Hypersim, which closely matches our indoor deployment setting. This provides strong generalization without additional training.

Finally, we observed that system-level performance improvements were more effectively achieved through:

- temporal smoothing (EMA)
- decision logic refinement

rather than modifying the model weights.

Therefore, given:

- strong quantitative performance ($\delta_1 \approx 0.909$)
- satisfactory real-world behavior
- and diminishing returns from additional training

We conclude that further fine-tuning is not required for this application.

6. Text-to-Speech Module

The Text-to-Speech (TTS) module is responsible for converting navigation commands into real-time spoken instructions for visually impaired users.

The system generates short, safety-critical commands such as:

- “Move left”
- “Move right”
- “Stop”
- “Obstacle ahead”

These commands must be:

- low latency
- highly intelligible
- reliable in offline environments

The TTS module operates as the final stage in the navigation pipeline asynchronously

6.1. **Design Requirements**

Based on the navigation use-case, the TTS system must satisfy:

Core Constraints

- Offline inference (no cloud dependency)
- Low latency (< 200 ms target)
- Clear pronunciation over naturalness
- Lightweight execution (edge deployable)

Why this matters

Unlike conversational AI, navigation systems are:

- time-critical
- action-driven
- error-sensitive

So clarity and speed dominate over expressiveness

6.2. **Candidate TTS Models**

We evaluated multiple TTS engines representing different trade-offs:

<u>Engine</u>	<u>Type</u>	<u>Strength</u>	<u>Weakness</u>
Piper	ONNX lightweight	Ultra-fast, CPU-friendly	Slightly robotic voice
Coqui TTS	Modular framework	Flexible models	Heavier
VITS-based models	Neural TTS	High naturalness	Higher latency
AI4Bharat Indic TTS	Multilingual neural	Produce best RTF in all compared model	Resource if we use multi language

Key Insight

This is not just a “best quality” problem — it’s a systems trade-off problem:

Latency vs Naturalness

CPU vs GPU

6.3. Evaluation Objectives

We evaluated TTS performance across three key dimensions:

Latency

Defined as:

Latency = Time from command generation → audio ready

Goal:

< 200 ms preferred

Must support real-time navigation

Speech Intelligibility

Measured using human evaluation:

% of commands understood in single listen

Focus:

clarity

pronunciation

no ambiguity

Multilingual Support

Currently support only Single language:

English

Goal:

Accurate pronunciation

No phonetic distortion

6.4. Benchmark Methodology

The evaluation pipeline followed a standardized approach:

Step 1: Command Set

A fixed command set was used:

Stop

Move left

Move right

Obstacle ahead

Turn slightly left

Door in front

Two steps forward

Step 2: Latency Measurement

Multiple runs per command

Mean + P95 recorded

Step 3: Real-Time Factor (RTF)

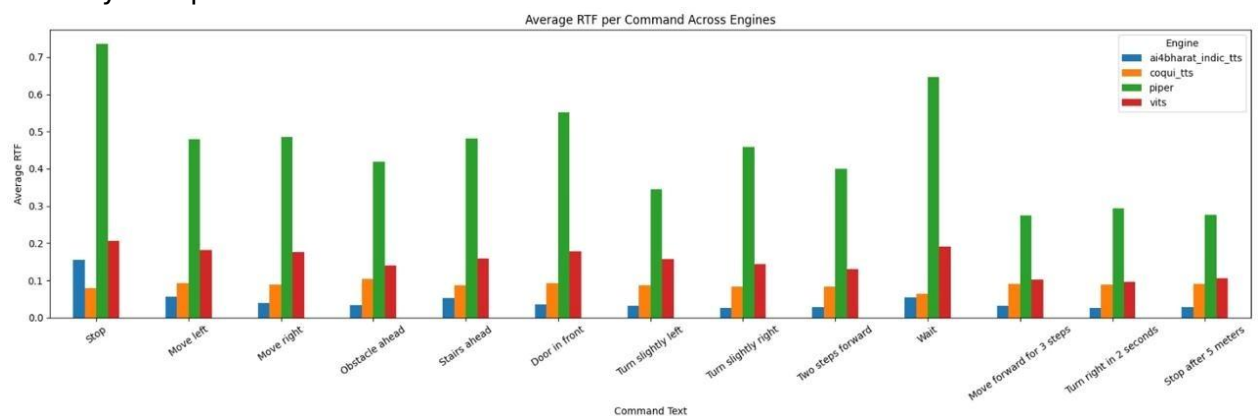
RTF = synthesis time/audio duration

Interpretation

RTF	Meaning

< 1	Faster than real-time
= 1	Real-time
> 1	Too slow

Experimental Results (Observed / Expected) Latency Comparison



Artifacts :

Jupiter notebook for evaluation

Future Work

Add adaptive speech rate (faster for urgency)

Add context-aware tone (alert vs normal)

Improve multilingual support using hybrid models

6.5. Justification for pretrain model:

- Improving TTS model weights give minimal system level gain
- Pretrain models already handle pronunciation perfectly
- Generalized across short phrases
- TTS fine tuning is normally used for Voice naturalness and pronunciation tweak as these are not the need of our system.